Packages and Interfaces

Java Packages

A **package** in Java is a **folder (directory)** that contains related Java classes, interfaces, enums, and sub-packages.

java
├── lang
├── util
├── io
└── net

Each package contains related classes.

Package	Contains
java.lang	String, Math, System, Object
java.util	Scanner, ArrayList, HashMap, Collections
java.io	File, BufferedReader, FileWriter
java.net	URL, Socket

Advantages of Packages

1. Organizes the Code
2. Avoids Class Name Conflicts
3. Provides Access Protection
4. Reusability

Types of Packages

There are two types**.**

1. Built-in Packages

These are already available in Java.

java.lang
java.util
java.io
java.sql
java.net

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your name:");
        String name = sc.nextLine();
        System.out.println(name);
    }
```

```
}
```

import java.util.*; //The * means "all classes in this package."

User-defined Package

You can create your own package.

```
package mypackage;
public class Student {
    public void display() {
        System.out.println("Student Class");
    }
}
```

```
import mypackage.Student;
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Packages and Access Specifiers

```
package college.student;

public class Student {
    public int a = 10;
    protected int b = 20;
    int c = 30;
    private int d = 40;
}
```

```
package college.faculty;

import college.student.Student;

public class Faculty {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.a);
    }
}
```

Java Interface

An **interface** in Java is a blueprint of a class that specifies **what a class must do**, but not **how it does it**.

```
interface Animal {
    void sound(); //public abstract void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
Dog Barks
```

Multiple Classes Implementing One Interface

```
interface Animal {
    void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}
class Cat implements Animal {
    public void sound() {
        System.out.println("Cat Meows");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
        d.sound();
        c.sound();
    }
}
```

Dog Barks
Cat Meows

Multiple Inheritance Using Interfaces

Java **does not support multiple inheritances with classes**.

This is illegal:

```
class A {
}
class B {
}
class C extends A, B {
} // Compilation Error.
```

```
class A {
    void display() {
        System.out.println("Class A");
    }
}

class B {
    void display() {
        System.out.println("Class B");
    }
}

class C extends A, B {
```

```
}

class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
    }
}
```

Ambiguity

```
interface A {
    void display();
}

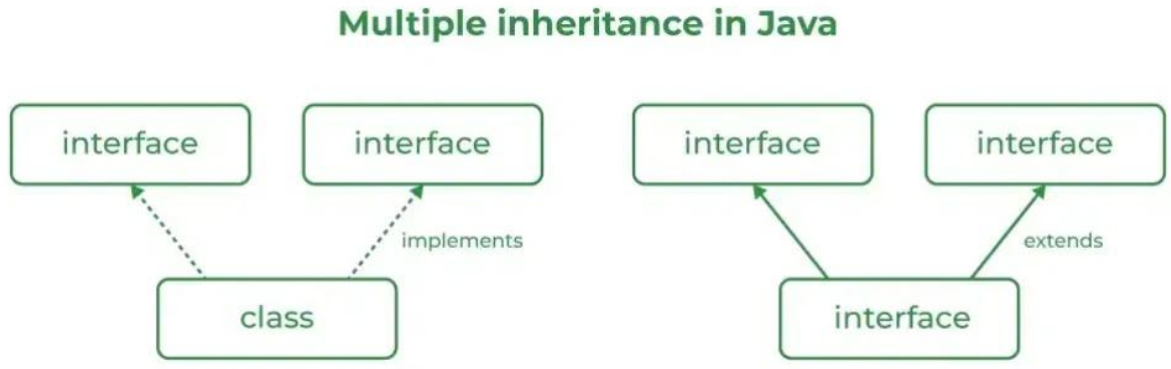
interface B {
    void display();
}

class C implements A, B {
    public void display() {
        System.out.println("Hello");
    }
}

class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
    }
}
```

Hello

But Java supports multiple inheritances through interfaces




```
interface A {
    void display();
}
interface B {
```

```
    void show();
}
class C implements A, B {
    public void display() {
        System.out.println("Display");
    }
    public void show() {
        System.out.println("Show");
    }
}
public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
        obj.show();
    }
}
```

Display
Show

```
import java.io.*;

interface Add{
    int add(int a,int b);
}

interface Sub{
        int sub(int a,int b);
}

class Cal implements Add , Sub
{       public int add(int a,int b){
        return a+b;
    }
        public int sub(int a,int b){
        return a-b;
    }
}

class Test{
    public static void main (String[] args){
        Cal x = new Cal();
        System.out.println("Addition : " + x.add(2,1));
        System.out.println("Substraction : " + x.sub(2,1));
    }
}
```

Addition : 3
Substraction : 1

```
interface A {
    void show();
}
```

```
interface B {
    void display();
}

interface C extends A, B {
    void print();
}

class Demo implements C {

    public void show() {
        System.out.println("Show");
    }

    public void display() {
        System.out.println("Display");
    }

    public void print() {
        System.out.println("Print");
    }
}

public class Main {
    public static void main(String[] args) {

        Demo d = new Demo();

        d.show();
        d.display();
        d.print();
    }
}
```

Show
Display
Print

Interface Variables

All variables in an interface are automatically:

- public
- static
- final

```
interface Demo {
    int VALUE = 100;
}
```

Java treats it as:

```
public static final int VALUE = 100;
```

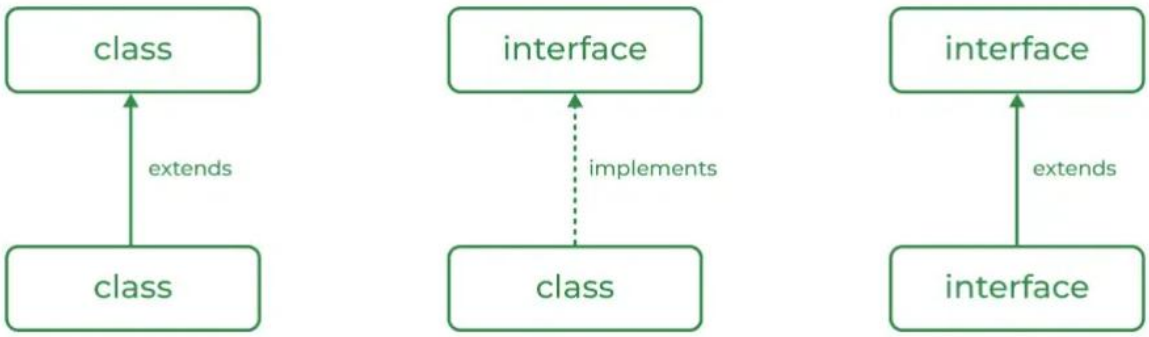
```
interface Demo {
    int VALUE = 100;
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Demo.VALUE);
    }
}
100
```

Demo.VALUE = 200; //Error because it is final.

Relationship between Class and Interface

A class can extend another class and similarly, an interface can extend another interface.

However, only a class can implement an interface and the reverse (an interface implementing a class) is not allowed.




```
interface Animal {
    void eat();
}

interface Dog extends Animal {
    void bark();
}

class Puppy implements Dog {

    public void eat() {
        System.out.println("Eating");
    }

    public void bark() {
        System.out.println("Barking");
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Puppy p = new Puppy();

        p.eat();
        p.bark();
    }
}
```

Eating
Barking

Default Methods

- Interfaces can define methods with default implementations.

```
interface Test
{
   default void display() {
   System.out.println("hello");
    }
}

class Main implements Test
{
        public static void main (String[] args) {
        Main obj = new Main();
        obj.display();
    }
}
```

Hello

Static Methods

- Interfaces can include static methods.
- These methods are called directly using the interface name and are not inherited by implementing classes.

```
interface Test
{
    static void display()
    {
        System.out.println("hello");
    }
}

class Main implements Test
{
    // display() can't be inherited here
    public static void main (String[] args)
```

```
    {
        Test.display();
    }
}
```
Hello

Private Methods

- Interface can now include private methods.
- Private methods are defined within the interface but it cannot be accessed by the implementing classes.
- Private methods cannot be overridden by implementing classes as they are not inherited.

```
interface Vehicle {
    // Private method for internal use
    private void startEngine() {
        System.out.println("Engine started.");
    }

    // Default method that uses the private method
    default void drive() {
        startEngine();
        System.out.println("Vehicle is now driving.");
    }
}

class Car implements Vehicle {
// startEngine () can't be inherited here
// Car class implements Vehicle interface and inherits the default method 'drive'
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        // This will call the default method, which in turn calls the private method
        car.drive();
    }
}
```

Functional Interface

A **Functional Interface** is an interface that contains **only one abstract method**.

It can have:

- ☑One abstract method
- ☑Any number of default methods
- ☑Any number of static methods
- ☑Any number of private methods
- ✗More than one abstract method

It is mainly used with **Lambda Expressions**.

```
@FunctionalInterface
interface Add {
    int sum(int a, int b);
}

public class Main {
    public static void main(String[] args) {

        Add obj = (a, b) -> a + b;
        System.out.println(obj.sum(10, 20));
    }
}
```

30